

Race Conditions

A simple, yet prominent, issue in concurrency

Daniel Rice

1 Definition

A race condition occurs when there is concurrent access to a shared resource, such as a program's critical section¹, causing a program's output/functionality to be dependent on uncontrollable events, for example due to clock-speed differences across cores accessing the critical section, or varying load across different cores/threads in a multithreaded program. A degree of concurrency, for the program in question, is required in order for there to be a possibility for (a) race condition(s).

2 Examples

2.1 Code-Example

To demonstrate a race condition, snippet 1 shows a program, which increments a global (reachable anywhere in the program) variable 1 million times (high number here to greater-display disparity), but also concurrently decrements the same counter 1 million times. After this the program outputs the final value of counter.

```
1 #include <iostream> //printing to stdout
2 #include <thread> //setting up threads
3
4 int counter = 0; //global var.
5
6 void increment() { //function to increment the global counter 1m. times; 1m. here to further
    ↪ demonstrate greater disparity from race condition inherently present here
7     for (int i = 0; i < 1000000; i++) { //1m. loop
8         counter++; //increment counter
9     }
10 }
11
12 void decrement() { //same as increment but will decrement the counter 1m. times
13     for (int i = 0; i < 1000000; i++) {
14         counter--;
15     }
16 }
17
18 int main() { //driver code to call each increment, decrement function on separate threads
19     std::thread thread1(increment);
20     std::thread thread2(decrement);
21     thread1.join(); //'join' forces whichever thread finished first to wait on the other
22     thread2.join();
23     std::cout << "Counter: " << counter << std::endl; //print the final val. of counter
24     return 0;
25 }
```

Snippet 1: Program displaying race condition, modifying a global counter.

¹Segment of data within a program, wherein shared-resources have the potential to be accessed by multiple threads concurrently.

The output from the program shown in snippet 1 (run 5 times) is observable in figure 1, showing the wildly differing values of counter per-time the program is run.

```
→ ~ ./raceCondition
Counter: -724922
→ ~ ./raceCondition
Counter: -838842
→ ~ ./raceCondition
Counter: 645949
→ ~ ./raceCondition
Counter: 125679
→ ~ ./raceCondition
Counter: 605913
```

Figure 1: Output of program from snippet 1 run 5 times

2.2 Producer-Consumer Problem

In this problem, there are two processes, producer and the consumer.

The producer's role is to fill a shared resource: a fixed-size buffer.

The consumer's role is to transfer data from the shared buffer elsewhere (perhaps elsewhere in the consumer's memory region).

Since the buffer is fixed-size (cannot grow/shrink), more data than can 'fit' in the buffer cannot be allowed (from the producer) (buffer overflow; undefined behaviour), further, more data than is in the buffer cannot be transferred from the buffer (by the consumer) (buffer underflow; undefined behaviour).

Henceforth, the solution must involve some synchronisation primitive to the buffer.

This problem is present due to the nature of concurrency, since if the producer is running on a different thread to the consumer, the two threads are likely not operating at identical clock-speeds (assuming threads running on different cores (CPUs)²), and even so, the load on each is likely to be different on an ordinary consumer-system³.

Furthermore, if each producer and consumer can access the buffer concurrently, the consumer may begin transferring data from the shared buffer before the producer has finished emplacing the data in the buffer; if this happens some (say half) of the data in the buffer have been 'consumed' by the consumer, but the remaining data may not, and this region may be marked as 'complete', or similar, as such there is some corruption of data shared between the two processes in this way. One solution to this, however, more solutions will be covered in Section 5, is for each the producer and consumer to 'signal' the other upon performing their respective operation, e.g. producer emplacing some data.

3 Causes

3.1 Concurrent Access to Shared Resources

Given some shared resources, e.g. a counter, as shown before, there is concurrent access, in snippet 1 there is concurrent access to the shared resource counter by each of the separate, concurrently executing, threads.

Concurrent access to a shared resource presents a race condition due to the way in which operations are performed - that is to say, without guaranteeing atomicity. For example, in snippet 1, control-flow may be alike the following:

(Let *T1* be thread 1; *T2* is thread 2.

Let *T1* run *increment()*; *T2* runs *decrement()*.)

1. *T1* reads counter's value into CPU register R1.

²However, even running on a machine with one-core, two-threads, race-conditions can still be present due to half a transaction occurring on *T1* then switching to *T2*, which are both modifying same data; not true concurrency, however atomicity is **not** guaranteed as a result of this.

³As opposed to a specialised-OS, wherein one process might be exclusively-allocated an entire core (CPU).

2. T1 computes $R1 + 1$ (increment operation), storing the result back in R1.
3. T2 reads `counter`'s value into CPU register R2.
4. T1 stores R1 back in `counter`.
5. T2 computes $R2 - 1$ (decrement operation), storing the result back in R2.
6. T2 stores R2 back in `counter`.

In the above sequence, the increment operation from T1 is lost, since it is overwritten by the value computed by T2, however crucially T2 computed its decremented value of `counter` before T1 had finished its computation **and** stored its result back in `counter`, as such the increment operation from T1 is no longer present in `counter`; this is known as a lost update.

This specific type of race condition (concurrent access to the same memory location, where at least one access is a write operation) is known as a data race.

3.2 Lack of Synchronisation

The program shown in snippet 1 could, in theory, print `counter`'s value to be 0, the chance would be astronomical, since the order of operations would have to be such that on all two-million total operations, each thread always reads the value of `counter` just after the other thread has written to `counter`; in one-million increments and decrements, one-million was chosen to ensure that even with astronomical-luck (in that perhaps even half the operations are perfectly synchronised), there will still be a large delta (from zero) in the final `counter`-output⁴. The solution is, therefore, to synchronise (write) accesses to the shared resource: `counter`.

4 Deadlocks

Deadlocks occur when a process, P1, is waiting for a resource held by another process, P2, which, in turn, is waiting for an additional resource to be released, currently held by P1.

This cyclical-waiting will result in indefinite waiting for each process; neither will terminate, unless a form of deadlock detection-and-recovery is employed at the OS-level⁵. Alternatively, this may be avoided using a form of deadlock avoidance, again implemented at the OS-level. Deadlock avoidance involves using Banker's Algorithm to determine a safe sequence of processes to run, such that the order the processes are run in will not result in a deadlock, as it is known how much of a given resource a given process will require ahead-of-time; in practice, however, this (maximum resources known ahead-of-time) is seldom true.

5 Solutions

Given the causes of race conditions, the global solution is any such, which prevents concurrent access to shared resources.

This can, however, greatly depend on the resource; for example, it may not be optimal to outright force all other threads/processes to wait until a lock on a shared resource is released (further, what happens if the lock is never released? - if improper-implementation is present, this can result in boundless-waiting, if this is present in, for example, process management in a kernel, the system will likely freeze eternally, requiring a hard-reset).

It may not be such that access to the shared resource is one-at-a-time (see 2.2 for the Producer-Consumer Problem), in which case multiple-access may be granted, up to some defined limit, in the case of producer-consumer problem this upper-limit would be the size of the buffer; in these instances it may be optimal to use a counting lock, wherein an atomic lock counter is incremented upon, e.g., another write (counting semaphore; see section 5.5.2).

⁴Zero since if doing the same number of increment and decrement operations, atomically, `counter` should be the same value: zero

⁵This may involve termination of a process.

5.1 Busy vs. Blocked Waiting

Busy waiting describes a synchronisation primitive in which attempted-access to a shared resource is constant, i.e the CPU is busy constantly checking if the lock has been released.

Synchronisation primitives which wait-busy initially seem wasteful, since the CPU cannot be used by other processes whilst waiting, and while this is true, for some operations, such as interrupt-handlers⁶, which have the highest priority (must have lowest response-time, therefore), cannot sleep, since they must run an operation as soon as the lock is released. Further, if the interrupt handler were to sleep, a context switch would be required to switch to another thread, however since interrupts do not operate on a schedule (and any queueing of interrupts, e.g. interrupting an interrupt, occurs pushing another frame onto the call-stack, each higher frame returning to the frame under), it may be such that the interrupt never executes, if sleeping were allowed, therefore.

Blocked waiting describes a synchronisation primitive in which access to a critical section is controlled by flags or other signals. This allows the thread to sleep until signalled by the thread that is currently holding the lock, at which point the thread can resume and carry out its operation in the critical section. With reference to the Producer-Consumer Problem, if the producer is filling the shared-buffer, the consumer thread can sleep (allowing it to be used by other processes) until it receives a signal from the producer indicating the producer has finished with the shared-resource (buffer), as such allowing the consumer to carry out its transfer (or other such) operation(s).

5.2 Peterson's Solution

Peterson's solution is one solution to synchronising concurrent-access to a shared-resource.

This works by two processes each having a turn variable, which indicates which of the two process' turn it is to enter the critical section. Further, a two-element global-Boolean-array, flag, allows each process to set True or False if it wishes to enter the critical section. Initially, the flag array will be set to False for both processes.

From this, a busy-waiting cycle checks turn, and if the process whose 'turn' it is to enter the critical section has also set True in flag, that process is allowed to enter the critical section.

However, this relies on the process entering the critical section to set False in flag after their operation has completed via access to the critical section, thereby risking starvation. Furthermore, this solution only works with two processes, extending this beyond two processes would even-greater increase the risk of starvation, as well as coordination issues, and general performance issues with this solution as a result of a simple flag array.

As such, this solution is not greatly used in modern systems-architecture.

5.3 Atomic Variables

Atomic variables take advantage of low-level Compare-and-Swap (CAS) operations to ensure when updating an atomic variable with a new value, the current value of the atomic variable is equal to the expected value, which is taken when first reading the atomic variable.

Applying this to the global-counter race-condition seen in snippet 1, if the counter was an atomic variable, when T1 wishes to increment counter, it first copies the current value, let this be 0, then once it has computed the value read incremented, it stores this back in counter, however first checking if counter is equal to the expected value, which was 0, if so, no lost-updates will occur from storing the result, 1 (0 incremented), in counter. If, however, counter does **not** equal the expected value of 0, this increment must be re-calculated.

Clearly, this is a very inefficient solution for the counter problem shown, since given the delta (from 0) of the program's output, seen in figure 1, a significant number of re-calculations would have to occur, due to counter storing a value different from the first-read, expected, value. As such, with this specific example, since there is significant concurrent-access to the critical section, many CPU cycles will be wasted if using atomic variables here⁷.

5.4 Mutexes

Mutexes are a locking mechanism which implements ownership on the mutex, such that only the thread which locked the mutex can release, or unlock, it. This is in contrast to semaphores (see below), which function more-so via

⁶Wherein a synchronisation-mechanism, such as a spinlock is necessary.

⁷It is worth mentioning that the program shown in snippet 1 is slower as a result of attempted-concurrency, since there is overhead for locking mechanisms, particularly with atomic variables, in this specific example; this problem was shown to demonstrate a race-condition in the simplest case. This program should instead run on a single-thread.

signalling⁸.

One issue with mutexes as a locking mechanism is the aforementioned deadlock-situation, which can occur if there is either/or no safe sequence of processes to run, or the mutexes are improperly used, e.g. not releasing a mutex after leaving the critical section.

Snippet 2 shows a program which employs a mutex to lock access to counter, which results in one-million increments occurring (since increment() is set-up first in thread1), then when decrement() runs on thread2 it must wait before counterMutex is released before it can access counter.

```
1 #include <iostream>
2 #include <thread>
3 #include <mutex> //mutex library
4
5 int counter = 0;
6 std::mutex counterMutex; //setup mutex for the counter
7
8 void increment() {
9     counterMutex.lock();//lock the mutex
10    for (int i = 0; i < 1000000; i++) {
11        counter++; //now increment 1m. times
12    }
13    std::cout << "Counter inside increment function: " << counter << std::endl;
14    counterMutex.unlock(); //releases mutex allowing other function to access counter var. (
    ↪ assuming this function was run first)
15 }
16
17 void decrement() {
18     counterMutex.lock();
19     for (int i = 0; i < 1000000; i++) {
20         counter--; //now decrementing 1m. times
21     }
22     std::cout << "Counter inside decrement function: " << counter << std::endl;
23     counterMutex.unlock();
24 }
25
26 int main() {
27     std::thread thread1(increment);
28     std::thread thread2(decrement);
29
30     thread1.join();
31     thread2.join();
32
33     std::cout << "Counter: " << counter << std::endl;
34
35     return 0;
36 }
```

Snippet 2: Program displaying mutexes to avoid a race condition, modifying a global counter.

```
→ ~ ./noRaceCondition
Counter inside increment function: 1000000
Counter inside decrement function: 0
Counter: 0
```

Figure 2: Output of program from snippet 2

⁸However, note that mutexes and **binary** semaphores are similar in kind.

The output of the program shown in snippet 2 can be seen in figure 2.

As explained in section 4, when working with locking mechanisms, particularly mutexes, if improperly implemented, this can result in a deadlock. To demonstrate a deadlock with a slightly-modified program from that shown in snippet 2, see snippet 3.

```
1 #include <thread>
2 #include <mutex>
3
4 int counterA = 0;
5 std::mutex counterAMutex; //setup mutex for the counter
6 int counterB = 0;
7 std::mutex counterBMutex;
8
9 void increment() {
10     counterAMutex.lock();//lock the mutex
11
12     for (int i = 0; i < 1000000; i++) {
13         counterA++; //now increment 1m. times
14     }
15     counterBMutex.lock();//wrongfully, before releasing counterAMutex, attempt to acquire
16     //↪ counterB
17     //this now depends on counterBMutex which will not be released until counterAMutex is
18     counterB++; // attempting to modify counterB, though this should be unreachable due to
19     //↪ the deadlock
20     counterBMutex.unlock();
21     counterAMutex.unlock(); //releases mutex allowing other function to access counter var.
22     //↪ (assuming this function was run first)
23 }
24
25 void decrement() {
26     counterBMutex.lock();
27     for (int i = 0; i < 1000000; i++) {
28         counterB--; //now decrementing 1m. times
29     }
30     counterAMutex.lock(); // deadlock
31     counterA++;
32     counterAMutex.unlock();
33     counterBMutex.unlock();
34 }
35
36 int main() {
37     std::thread thread1(increment);
38     std::thread thread2(decrement);
39
40     thread1.join();
41     thread2.join();
42
43     return 0;
44 }
```

Snippet 3: Modified version of snippet 2, displaying a deadlock.

In snippet 3, due to each thread attempting to acquire each other's mutex before each has released theirs results in endless waiting; deadlock.

5.5 Semaphores

Semaphores are another synchronisation primitive, and have two variants: binary; and counting.

5.5.1 Binary

Binary semaphores have two operations `wait` and `signal`.

Binary semaphores are atomic variables, and either store 0 or 1, henceforth this only allows single-access to a shared-resource.

This is similar to Peterson's Solution, however slightly less specific, as there is no `turn` variable or `flag` array; semaphores operate via `wait` and `signal` signals, as such allows multiple processes to wait, then wake-up when they are signalled (once the critical section is free). Depending on the implementation of the binary semaphore, the queuing may be some algorithm such as FCFS, potentially allowing a priority to be specified (perhaps then implementing ageing⁹ to prevent total-starvation).

5.5.2 Counting

All previous synchronisation primitives discussed thus-far have only allowed one process to hold a shared-resource at a time, however, counting semaphores, by contrast, can allow multiple processes to hold the same resource. This may seem as though race conditions are guaranteed if using counting semaphores, however, it is for this reason that this synchronisation method is only applicable to particular tasks.

With reference to the Producer-Consumer Problem, the counting semaphore may store the filled slots in the buffer, such that the producer increments the semaphore upon adding an item to the buffer, with the consumer decrementing the semaphore upon transferring-out an item from the buffer. Note that data may still be overwritten in the buffer, as such a mutex is required for accessing the shared buffer, with a second semaphore being to track the number of empty slots. This is more efficient than only using a mutex, since else both the producer and consumer would have to acquire the mutex in order to check if the buffer is full/empty, respectively for the producer and the consumer. In this way, busy-waiting is avoided, since one of the threads can sleep until signalled by the other, then acquiring the mutex and producing or consuming items to/from the buffer. As a result of blocked-waiting, this scales significantly-better (having many producers and consumers) than using only a mutex on the critical section. ([GeeksforGeeks \(2024\)](#)).

6 Race-Condition Vulnerability

As explained, race conditions can cause inconsistencies in data (incorrect or corrupt data), however they can also present themselves as a vulnerability to the program/system they are present within. For example, in a TOCTOU¹⁰ attack, the time between checking some data and using it differ (the race window), in the case of a race-condition being present, this may allow modification of the data to-be-fetched after the check, but before the time it is to be used. If this is an address to jump/return to, and is overwritten by way of this attack, it may be possible that control-flow could be hijacked, by setting another thread to concurrently the address (if locks or other synchronisation techniques are not present in the program checking and using the address) ([GeeksforGeeks \(2025\)](#)).

6.1 Example

[I Code 4 Coffee: Hacking the Xbox 360 Hypervisor Part 2: The Bad Update Exploit \(Miceli \(2025\)\)](#) is a write-up of Ryan Miceli's (otherwise known as 'Grimdoomer') attempt at defeating the Xbox 360's hypervisor via software-only techniques.

The attack vector devised was vastly complex, containing many crucial components, without which this vector would be infeasible, however one crucial component was a race-condition vulnerability, allowing Grimdoomer to decompress an XKE payload ('Bootloader Update Payload') to a specified address in memory, stored in the `dec_output_buffer` pointer, wherein another thread was created 'hammering the `dec_output_pointer` to point into the last segment of the hypervisor', this alone would not allow for code execution in the hypervisor, however does allow overwriting, in this instance the last segment of the hypervisor, however this could instead be anywhere in the hypervisor to cause memory corruption in the hypervisor, with a modified XKE payload.

⁹Increasing a process' priority the longer it waits.

¹⁰Time-Of-Check, Time-Of-Use.

References

GeeksforGeeks (2024), 'Producer consumer problem using semaphores | set 1'.

URL: www.geeksforgeeks.org/operating-systems/producer-consumer-problem-using-semaphores-set-1/

GeeksforGeeks (2025), 'Race condition vulnerability'.

URL: www.geeksforgeeks.org/operating-systems/race-condition-vulnerability/

Miceli, R. (2025), 'Hacking the xbox 360 hypervisor part 2: The bad update exploit'.

URL: icode4.coffee/?p=1081